

Positive testing: Unary operations

Test 1 — GetMovieDetail (positive)

Endpoint: ws://localhost:8080/ws/movies/detail

Request sent:

```
{"movieId": 1}
```

Expected response: The server returns a MovieDetailResponse JSON object containing the full movie details for Inception, including a genres array.

```
{
  "id": 1,
  "title": "Inception",
  "releaseYear": 2010,
  "runtimeMinutes": 148,
  "director": "Christopher Nolan",
  "synopsis": "...",
  "genres": ["Action", "Sci-Fi"]
}
```

Result:

The screenshot shows a websocket testing tool interface. At the top, the URL bar displays 'ws://localhost:8080/ws/movies/detail'. Below the URL bar, there are tabs for 'Docs', 'Message', 'Params', 'Headers', and 'Settings'. The 'Message' tab is active, showing a message box with the text '{"movieId": 1}' and a 'Send' button. To the right of the message box, there is a 'Disconnect' button. Below the message box, there is a 'Response' section. The 'Response' section shows a list of messages. The first message is '{"director": "Christopher Nolan", "genres": ["Action", "Sci-Fi"], "id": 1, "releaseYear": 2010, "r...' with a timestamp of '00:25:38.123'. The second message is '{"movieId": 1}' with a timestamp of '00:25:38.118'. There is a 'Restore' button at the bottom right of the response section. The interface also includes a 'Save' button, a 'Share' button, and a 'Cookies' section on the right side.

Negative testing: Unary operations

Test 2 — GetMovieDetail (malformed JSON)

Endpoint: `ws://localhost:8080/ws/movies/detail`

Request sent:

`abc`

Expected response: A structured JSON error envelope:

```
{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}
```

The connection stays open — the client can send another valid request on the same connection.

Result:

The screenshot shows a websocket testing tool interface. At the top, the URL `ws://localhost:8080/ws/movies/detail` is entered. Below the URL bar, there are tabs for `Docs`, `Message`, `Params`, `Headers`, and `Settings`. The `Message` tab is selected, showing the message `abc` sent. To the right of the message input, there is a `Disconnect` button. Below the message input, there is a `Send` button. The `Response` section shows a list of messages received. The first message is a JSON error response: `{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}`, received at `00:27:18.184`. The second message is `abc`, received at `00:27:18.180`. The third message is `Connected to ws://localhost:8080/ws/movies/detail`, received at `00:27:17.212`. The status bar at the bottom shows `Connected` and `Save Response` button.

Test 3 — GetMovieDetail (not found)

Endpoint: ws://localhost:8080/ws/movies/detail

Request sent:

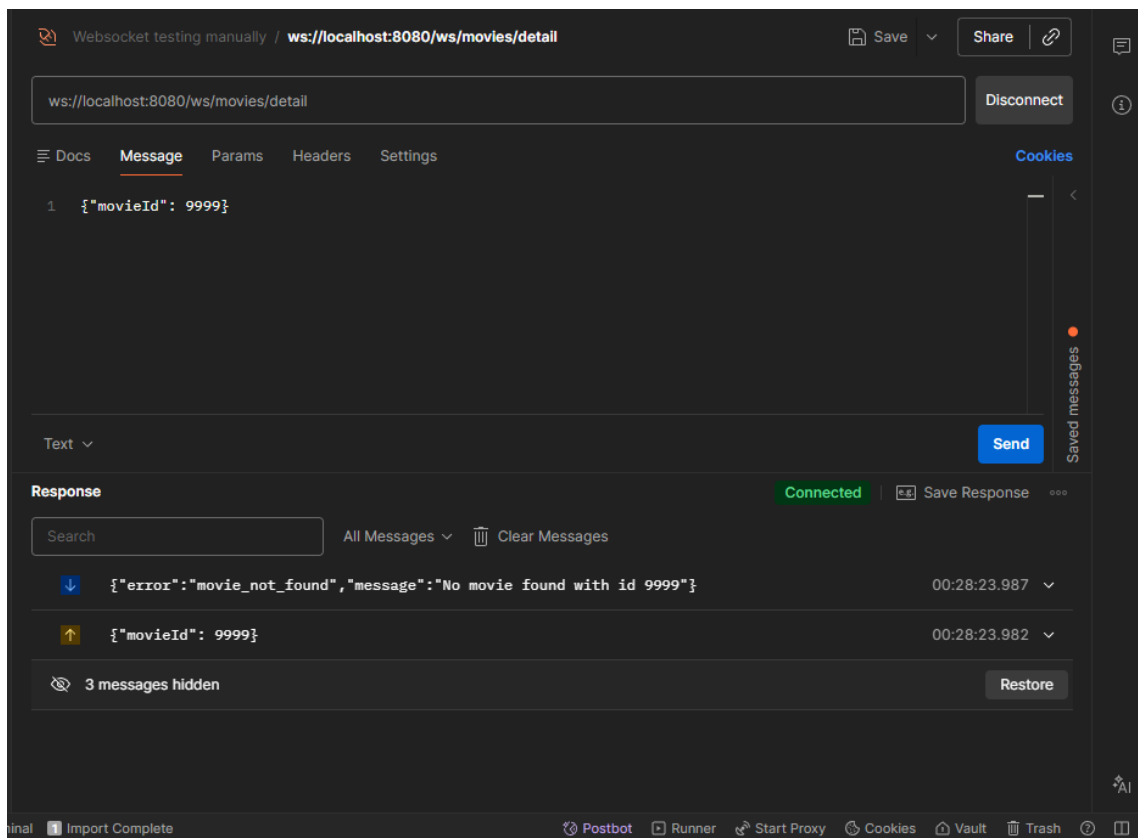
```
{"movieId": 9999}
```

Expected response: A structured JSON error envelope:

```
{"error": "movie_not_found", "message": "No movie found with id 9999"}
```

The connection stays open.

Result:



Positive testing: Bidirectional Streaming operations

Test 4 — ReviewStream (subscribe and receive reviews)

Endpoint: ws://localhost:8080/ws/movies/stream

Request sent:

```
{"movieId": 1}
```

Expected response: The server immediately streams all existing reviews for movie 1 as individual ReviewResponse JSON frames. Each frame contains reviewId, movieId, movieTitle, rating, comment, and createdAt.

Movie 1 (Inception) has 4 reviews in the database (ids 101, 102, 103, 111), so we expect 4 frames. Example first frame:

```
{
  "reviewId": 101,
  "movieId": 1,
  "movieTitle": "Inception",
  "rating": 9,
  "comment": "A masterpiece of layered storytelling.",
  "createdAt": "..."}
}
```

The connection stays open. The server continues polling every ~2 seconds for new reviews.

Result:

The screenshot shows a websocket testing tool interface. At the top, the URL 'ws://localhost:8080/ws/movies/stream' is entered. Below the URL bar, the 'Message' tab is active, showing the request: `{ "movieId": 1 }`. The 'Response' tab is also active, showing a list of received messages. The first message is `{ "comment": "<script>alert('xss')</script>", "createdAt": "2026-06-01T..." }`. The second message is `{ "comment": "Rewatchable every single year.", "createdAt": "2026-06-01T22:23:16", "movieId": ... }`. The third message is `{ "comment": "Ambitious but the ending is too ambiguous.", "createdAt": "2026-06-01T22:23:16", "movieId": ... }`. The fourth message is `{ "comment": "A masterpiece of layered storytelling.", "createdAt": "2026-06-01T22:23:16", "movieId": ... }`. The fifth message is `{ "movieId": 1 }`. The status bar at the bottom indicates 'Connected to ws://localhost:8080/ws/movies/stream'.

Test 5 — ReviewStream (add second subscription on same connection)

Endpoint: `ws://localhost:8080/ws/movies/stream` (same connection as Test 4)

Request sent (on same connection):

```
{"movieId": 2}
```

Expected response: The server streams all existing reviews for movie 2 (The Dark Knight) on the same connection. Movie 2 has 3 reviews in the database (ids 104, 105, 106), so we expect 3 additional frames. Both movie 1 and movie 2 reviews now flow on this connection.

Result:

The screenshot shows a websocket testing tool interface. At the top, the URL `ws://localhost:8080/ws/movies/stream` is entered. Below the URL bar, there are tabs for 'Docs', 'Message', 'Params', 'Headers', and 'Settings'. The 'Message' tab is active, showing a single message: `1 {"movieId": 2}`. To the right of the message input, there is a 'Send' button. Below the message input, there is a 'Text' dropdown menu. On the right side of the interface, there is a 'Disconnect' button and a 'Cookies' section. Below the message input, there is a 'Response' section. The 'Response' section shows a list of messages received from the server. The messages are as follows:

- `1 {"comment": "Excellent, though the third act drags slightly.", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `2 {"comment": "A crime thriller that happens to feature Batman.", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `3 {"comment": "Best superhero movie ever made.", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `4 {"movieId": 2}`
- `5 {"comment": "<script>alert('xss')</script>", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `6 {"comment": "Rewatchable every single year.", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `7 {"comment": "Ambitious but the ending is too ambiguous.", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `8 {"comment": "A masterpiece of layered storytelling.", "createdAt": "2026-06-01T22:23:16", "movieId": 2}`
- `9 {"movieId": 1}`

The interface also shows a 'Connected' status and a 'Save Response' button. At the bottom, there is a status bar with various icons and text: 'Import Complete', 'Postbot', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and a help icon.

Test 6 — ReviewStream (live push — add review while subscribed)

Endpoint: `ws://localhost:8080/ws/movies/stream` (same connection as Test 4/5)

While the stream connection is open and subscribed to movie 1, a new review is added

Expected response: Within ~2 seconds, the newly added review appears as a `ReviewResponse` frame on the WebSocket connection — without the client sending any message.

Result:

The screenshot shows a WebSocket testing tool interface. At the top, the URL `ws://localhost:8080/ws/movies/stream` is entered. Below the URL bar, there are tabs for 'Docs', 'Message', 'Params', 'Headers', and 'Settings'. The 'Message' tab is selected, showing a single message: `1 {"movieId": 2}`. To the right of the message input area is a 'Send' button. Below the message input area, there is a 'Response' section. The status 'Connected' is shown in green. Below the status, there is a search bar and a 'Clear Messages' button. A list of messages is displayed, each with a timestamp and a JSON object. The messages are:

- `00:38:25.913` `{"comment": "Added during live stream test", "createdAt": "2026-06-01T22:38:25", "movieId": 1, "movieTitle": "..."} (highlighted with a red background)`
- `00:37:06.701` `{"comment": "Excellent, though the third act drags slightly.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "..."} (highlighted with a blue background)`
- `00:37:06.700` `{"comment": "A crime thriller that happens to feature Batman.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "..."} (highlighted with a blue background)`
- `00:37:06.699` `{"comment": "Best superhero movie ever made.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": "..."} (highlighted with a blue background)`
- `00:37:06.689` `{"movieId": 2}` (highlighted with a yellow background)
- `00:35:48.506` `{"comment": "<script>alert('XSS!');</script>", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "..."} (highlighted with a blue background)`

At the bottom of the interface, there is a status bar with various icons and labels: 'Postbot', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and 'AI'.

Negative testing: Bidirectional Streaming operations

Test 7 — ReviewStream (malformed JSON)

Endpoint: `ws://localhost:8080/ws/movies/stream`

Request sent:

abc

Expected response: A structured JSON error envelope:

```
{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}
```

The connection stays open — the client can send a valid subscription on the same connection.

Result:

The screenshot shows a websocket testing tool interface. At the top, the URL `ws://localhost:8080/ws/movies/stream` is entered. Below the URL bar, the 'Message' tab is active, showing the request 'abc'. The 'Response' tab is also visible, showing a JSON error message: `{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}`. The interface includes a 'Send' button and a 'Disconnect' button. The status bar at the bottom indicates 'Connected'.

Direction	Message	Timestamp
Down	<code>{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}</code>	00:40:06.625
Up	abc	00:40:06.620
Down	<code>{"comment": "Added during live stream test", "createdAt": "2026-06-01T22:38:25", "movieId": 1, "movieTitle": ...}</code>	00:38:25.913
Down	<code>{"comment": "Excellent, though the third act drags slightly.", "createdAt": "2026-06-01T22:23:16", "movieId": ...}</code>	00:37:06.701
Down	<code>{"comment": "A crime thriller that happens to feature Batman.", "createdAt": "2026-06-01T22:23:16", "movieId": ...}</code>	00:37:06.700
Down	<code>{"comment": "Best superhero movie ever made.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": ...}</code>	00:37:06.699

Test 8 — ReviewStream (not found)

Endpoint: ws://localhost:8080/ws/movies/stream

Request sent:

```
{"movieId": 9999}
```

Expected response: A structured JSON error envelope:

```
{"error": "movie_not_found", "message": "No movie found with id 9999"}
```

The connection stays open.

Result:

The screenshot displays a websocket testing application interface. At the top, the URL `ws://localhost:8080/ws/movies/stream` is entered in the address bar. Below the address bar, there are tabs for `Docs`, `Message`, `Params`, `Headers`, and `Settings`. The `Message` tab is active, showing a single message: `1 {"movieId": 9999}`. To the right of the message input area, there is a `Send` button. Below the message input area, the `Response` section is visible, showing a `Connected` status. The response messages are listed in a table with columns for message content and timestamp. The first message is `{"error": "movie_not_found", "message": "No movie found with id 9999"}` with a timestamp of `00:40:51.281`. The second message is `{"movieId": 9999}` with a timestamp of `00:40:51.277`. Below the response messages, there is a `Restore` button. The bottom of the interface shows a status bar with various icons and text, including `Import Complete`, `Postbot`, `Runner`, `Start Proxy`, `Cookies`, `Vault`, `Trash`, and `AI`.